

README

Highlights
Architecture
How marrow compares
Conformance
Install
Quickstart
Usage
Examples
Tests
Project status
License
Contributing
Acknowledgements



**The native runtime — and compiler — that runs an agent, below
MCP and A2A.**

*Reference implementation of **ARI**, the Agent Runtime Interface.
Embeddable in C++; authored in Python or TypeScript.*

marrow is two things on one native core:

1. A native **C++17 agent runtime** — the loop that holds conversation state, calls a model, dispatches tools, and routes messages between agents — with ergonomic Python bindings. It is the **reference implementation of ARI**, a language-neutral contract for the agent *runtime* layer.
2. An early **AI-aware compiler**: author an agent graph (in Python or TypeScript), and marrow lowers it into an **ARI manifest**, compiles that to a deterministic **RuntimePlan**, and executes it through the native runtime — with pluggable providers, a policy-gated tool-use loop, budgets, policy enforcement, rollback, and a portable **ExecutionTrace**.

Why this exists. The mature Python frameworks (LangGraph, CrewAI, AutoGen) are excellent for server apps, but they're Python-locked by design — which rules them out of a fast-growing class of deployments:

- **Embodied / robotic** systems with hard real-time constraints (no GC pauses).
- **Edge / on-device** deployments — constrained hardware, often offline, local models.
- **Native applications** — game engines, audio/DSP, trading systems — that need an agent loop without shipping a Python runtime to the customer.

marrow targets exactly that lane: a native core you can **embed without a managed runtime**, with bounded memory and no hot-path allocation (the **ARI-Embedded** profile). The C++ core compiles as a standalone library; the Python binding is a convenience, not a requirement. Real providers (OpenAI, Anthropic, Ollama) ship as thin Python subclasses that hook back into the core through a trampoline.

ARI sits *beneath* the agent protocol stack — it complements MCP and A2A, it does not compete with them:

A2A — agent-to-agent messaging (across hosts)
MCP — tool / context exchange (model ↔ world)
ARI — the runtime that runs a turn ← marrow implements this
 (state · provider · tools · routing)
Model provider (OpenAI / Anthropic / local)

And the compiler lowers an authored agent graph onto that runtime — a portable, content-addressed pipeline:

AgentGraph → ARI manifest → RuntimePlan → native runtime → ExecutionTrace
(Python/TS) (JSON) (JSON) (C++)
(JSON)

See [ARI-SPEC.md](#) for the contract and [docs/ari-strategy.md](#) for how the standard is meant to grow.

Project status: 0.1.0, public API frozen per [STABILITY.md](#). The C++ core is buildable, tested, and works on Linux / macOS / Windows × Python 3.9–3.12. Production primitives — timeouts, cancellation, bounded inboxes, persistence, tracing, usage tracking — all shipped. Real-world soak testing and bus-factor-of-2 are the remaining items before this should be your default choice for a live system. See [ROADMAP.md](#).

Benchmarks — see [benchmarks/](#). M-series Mac, Python 3.12: ~1.4M AgentState.append/s, ~672K router send+drain/s, ~64K full 3-agent pipeline iterations/s with MockProvider. Run `python -m benchmarks.compare_langgraph` for a head-to-head against LangGraph on a 1-node echo graph (`pip install '[bench]'`).

Highlights

Compiler (early, feature-complete):

- **AI-aware compiler** — author an AgentGraph, compile to an ARI manifest → deterministic RuntimePlan → execute → portable ExecutionTrace. Content-addressed ids make the whole chain reproducible and replayable.
- **Governance, enforced** — policy checkpoints (allow / deny / require_approval, with an approver and evidence), budgets (steps / tokens / cost / wall-clock), rollback on failure, and deterministic replay — all recorded in the trace.
- **Policy-gated tool use** — agents request tools, the call is gated by tool: policy and restricted to the agent's declared tools, invoked through the C++ registry, and the result is fed back.
- **Pluggable providers** — mock (keyless default), openai / anthropic / ollama built in, or bring-your-own via a factory.
- **Multi-language frontends** — a Python frontend and a [TypeScript frontend](#) that emits **byte-identical** ARI (verified by a cross-language parity test).

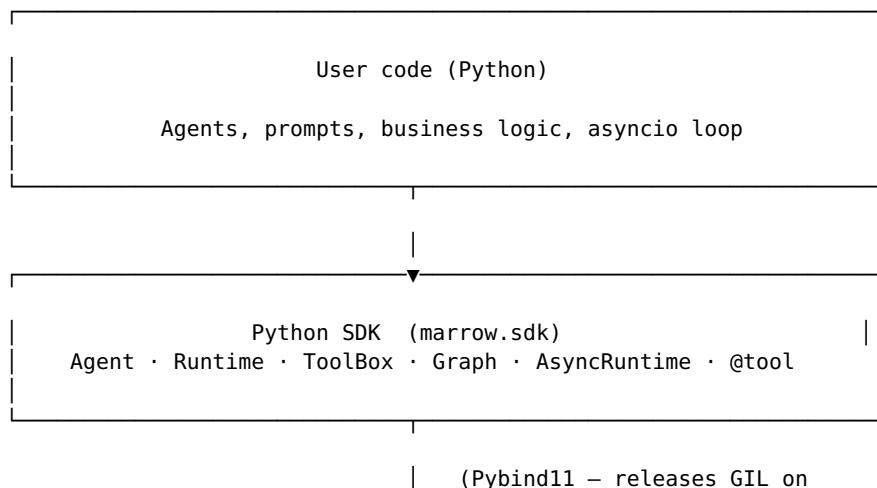
Runtime (ARI reference implementation):

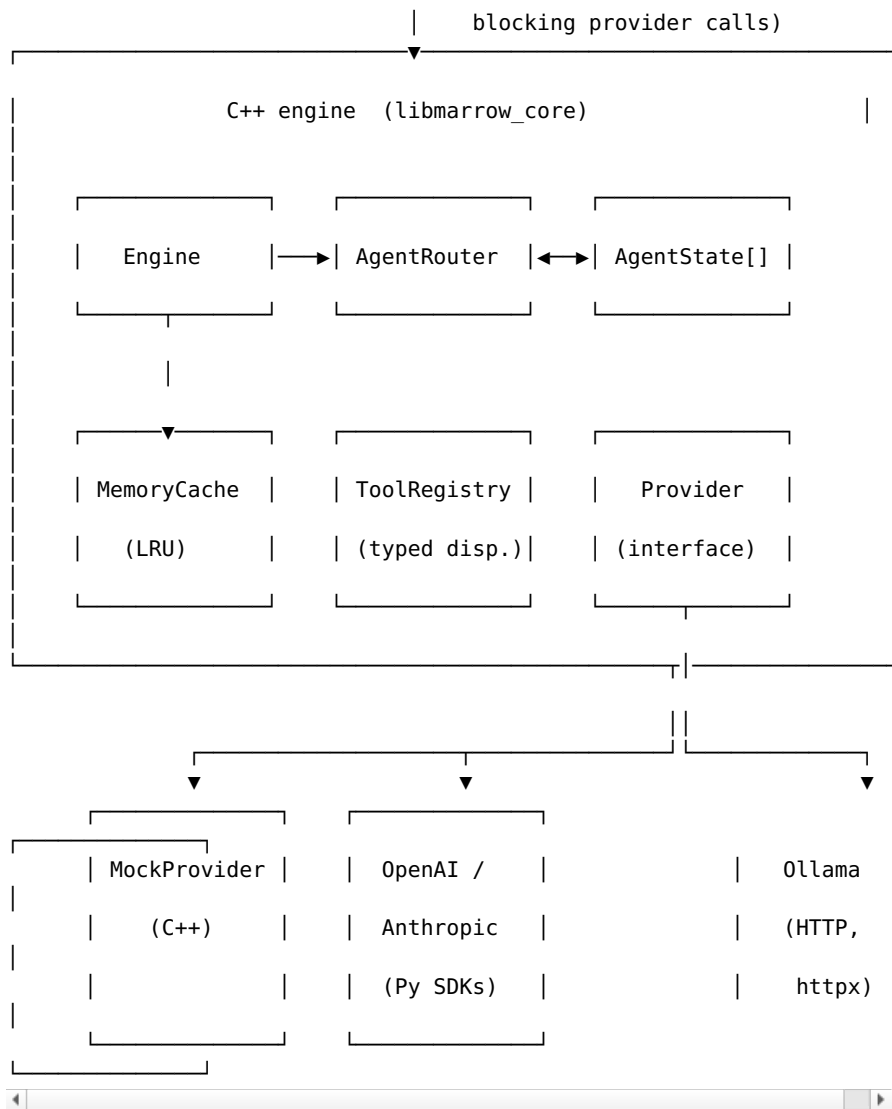
- **Native C++ core** — Message history, LRU cache, agent router, tool registry, CancelToken. C++17, std::shared_mutex for read-heavy

access, no third-party C++ deps beyond Pybind11.

- **Ergonomic Python** — Agent + Runtime dataclass API. Two lines from import to your first generated message.
- **Real providers** — OpenAIProvider, AnthropicProvider (with prompt caching), OllamaProvider. Adding a new one is a ~30-line subclass.
- **Streaming first-class** — Every provider implements generate_stream. Agent.stream(on_chunk=...) works the same way regardless of vendor.
- **Per-call timeouts + cancellation** — Pass timeout_ms= and a CancelToken to any step() or stream() call.
- **Backpressure** — Router.set_inbox_limit(agent_id, max_size, policy) with three policies (Reject / DropOldest / DropNewest).
- **Graceful shutdown** — Runtime.shutdown() blocks new agent creation; in-flight work finishes.
- **Persistence** — StateStore interface with InMemoryStateStore and SQLiteStateStore. Restart your app, resume your conversations.
- **Observability** — TraceSink Protocol with NullTraceSink / PrintTraceSink / OpenTelemetryTraceSink. Structured logging via marrow.logging_config.
- **Usage tracking + cost estimation** — UsageTracker aggregates tokens per agent + model with configurable pricing tables.
- **Retry + rate limiting** — RetryPolicy (exponential backoff with jitter) and RateLimiter (token bucket); configure once on the Runtime and every agent inherits them.
- **Tools dispatched from C++** — Registry lookup and dispatch in native code; tool bodies stay in Python. Tool errors are redacted; args and results have configurable size caps.
- **Fluent graphs, not YAML** — Graph().start().then().finish(). Returns a structured GraphResult so callers can distinguish "reached end" from "ran out of steps" — no silent truncation.
- **Asyncio bridge** — AsyncRuntime wraps the C++ engine in a ThreadPoolExecutor. Pybind11 bindings release the GIL on Provider::generate and generate_stream, so threaded provider calls scale.
- **Thread-safe core** — Engine, AgentRouter, AgentState, MemoryCache, ToolRegistry verified by tests/test_concurrency.py. CI runs the suite under ThreadSanitizer (informational).
- **Embeddable from C++** — The core compiles as a static library independent of Pybind11. See [examples/embed_cpp/](#) for a pure-C++ demo.
- **Stability promise** — Public API stable per [STABILITY.md](#) with documented semver + deprecation flow.
- **Tiny dependency footprint** — Core requires only Pybind11. Real providers are opt-in extras.

Architecture





What lives where

Layer	Responsibilities	Files
C++ core	Message, AgentState, MemoryCache, AgentRouter, ToolRegistry, Engine, Provider interface	src/core/engine.{hpp,cpp}
Bindings	Pybind11 module, PyProvider trampoline, GIL release on hot paths	src/bindings/bindings.cpp
Python SDK	Agent, Runtime, Graph, ToolBox, AsyncRuntime	python/marrow/
Providers	OpenAIProvider, AnthropicProvider, OllamaProvider (optional extras)	python/marrow/providers/

See [docs/design-notes.md](#) for the design rationale and deliberate trade-offs.

How marrow compares

	marrow	LangGraph	CrewAI	AutoGen
Native hot	C++17	Python	Python	Python

path				
GIL released during providers	yes	partial	no	no
Streaming	built-in	yes	yes	yes
Tool dispatch	C++ registry	Python	Python	Python
Graph builder	fluent Python	fluent Python	declarative	declarative
Required dependencies	Pybind11 only	many	many	many
Multi-agent concurrency	threads + GIL release	asyncio	sequential	asyncio
Embeddable from C++	yes	no	no	no
Status	pre-alpha	stable	stable	stable

Read this honestly: LangGraph / CrewAI / AutoGen are mature, supported, and have far more features today, and for a server-side Python app you should probably use one of them. marrow is not trying to win that comparison on features or on raw speed — if your bottleneck is network I/O (it usually is, server-side), the speedup is modest.

The row that matters is the last one: **Embeddable without a Python runtime**. That is a structural property the others can't offer without ceasing to be Python frameworks — and it's the whole point. If you're putting an agent loop inside a **robot, a drone, an edge device, or a native C++ application**, the Python frameworks aren't an option and marrow is. That lane — the **ARI-Embedded** profile — is what marrow is built to own.

Conformance

marrow is the reference implementation of [ARI](#), and it's held to the spec by an executable **conformance kit** in [ari-conformance/](#). Each test maps to a numbered ARI requirement, so "conformant" is *falsifiable*, not a marketing claim.

```

pip install -e ".[test]"
pytest ari-conformance/ -v          # full kit (needs the built
extension)
pytest ari-conformance/test_ari_embedded_structural.py -v #
structural checks, no build

```

The kit runs in CI on every push (conformance job). When making an ARI conformance claim, name the profile + version per [ARI §10.4](#), e.g. "ARI 0.1 · Core + Embedded · kit v0.1."

Install

```

python -m venv .venv && source .venv/bin/activate
pip install -U pip scikit-build-core pybind11 cmake
pip install -e .

```

With real providers:

```

pip install 'marrow-rt[openai]'      # OpenAI Chat Completions
pip install 'marrow-rt[anthropic]'  # Anthropic Messages + prompt

```

caching

```
pip install 'marrow-rt[ollama]'      # local Ollama daemon
pip install 'marrow-rt[all]'         # everything above
```

The wheels.yml workflow builds Linux / macOS / Windows x Python 3.9-3.12 wheels and uploads them as GitHub artifacts on v* tag pushes. PyPI publishing is **not** yet wired up; install via git or the artifact downloads until v0.1.

Quickstart

```
from marrow import Agent, Runtime, MockProvider

rt = Runtime()
provider = MockProvider()

researcher = rt.add(Agent("researcher", provider,
system_prompt="Research."))
writer = rt.add(Agent("writer", provider,
system_prompt="Write."))

researcher.append_user("Find three facts about graph databases.")
findings = researcher.step()

rt.handoff(frm="researcher", to="writer", text=findings)
rt.deliver(writer)
print(writer.step())
```

Usage

Real providers

```
from marrow import Agent, Runtime
from marrow.providers import AnthropicProvider

rt = Runtime()
provider = AnthropicProvider(model="claude-sonnet-4-6") # caches
system prompt
agent = rt.add(Agent("assistant", provider, system_prompt="Be
concise."))

agent.append_user("What is a graph database?")
print(agent.step())
```

Streaming

```
agent.append_user("Write a haiku about graph databases.")
agent.stream(on_chunk=lambda c: print(c, end="", flush=True))
```

Works identically for OpenAIProvider, AnthropicProvider, OllamaProvider, MockProvider, and any Python subclass of PyProviderBase.

Tools

```
from marrow import Runtime, ToolBox, tool

@tool(description="Add two integers")
def add(a: int, b: int) -> int:
    return a + b
```

```

rt = Runtime()
ToolBox().add(add).bind(rt)           # registered in the C++
registry

print(rt.tools.invoke("add", '{"a": 2, "b": 3}'))
# {"ok": true, "result": 5}

```

JSON-schema is auto-generated from annotations for the common cases — primitives, Optional[X], Union, list[X], dict, Literal[...]. Stored as a string alongside the tool in C++. For complex types (Pydantic models, dataclasses) pass `schema=` explicitly on `@tool` rather than relying on the auto-generator.

Multi-agent graphs

```

from marrow import Graph, run_graph

g = (Graph()
     .start("researcher")
     .then("researcher", "writer")
     .then("writer", "editor", when=lambda s: "DRAFT" in s)
     .finish("editor"))

final = run_graph(rt, agents_by_name, g, initial_input="Topic:
graphs.")

```

Fluent, introspectable, no YAML. See [design-notes](#) for why.

Async / concurrency

```

import asyncio
from marrow import AsyncRuntime, Agent, MockProvider, Runtime

async def main():
    rt = AsyncRuntime(Runtime())
    agents = [rt.add(Agent(f"w{i}", MockProvider())) for i in
range(8)]
    for i, a in enumerate(agents):
        a.append_user(f"hello from worker {i}")
    results = await rt.gather_steps(agents)  # concurrent

asyncio.run(main())

```

The C++ binding releases the GIL during `Provider::generate`, so threaded steps make real parallel progress while their Python provider code runs.

Examples

All runnable. From the repo root after `pip install -e .`:

File	Demonstrates
examples/main.py	Two-agent handoff
examples/tools_example.py	Tool registry, schema generation, invocation
examples/graph_example.py	Three-node graph execution
examples/streaming_example.py	Word-by-word streaming
examples/async_example.py	Concurrent agent steps
examples/openai_example.py	OpenAI provider (requires OPENAI_API_KEY)
examples/anthropic_example.py	Anthropic provider with prompt caching

Tests

```
pip install -e ".[test]"
pytest -v
```

Current suite: ~20 tests across smoke, router edge cases, tool registry (including args/result size caps and error redaction), streaming (Python-subclassed Provider trampoline path), and a dedicated concurrency suite that exercises the `shared_mutex` paths under contention.

Project status

This is **0.1.0** — the public API is frozen per [STABILITY.md](#). The C++ core builds and runs on Linux / macOS / Windows × Python 3.9-3.12. The architecture has been pressure-tested and the controversial trade-offs are documented in [docs/design-notes.md](#).

Against the [ARI](#) profiles, marrow currently targets **ARI-Core** and **ARI-Embedded** (with **ARI-Server** features present but not yet soaked-tested).

What is shipped and stable:

- Core message / state / cache / router APIs (`std::shared_mutex`, read-heavy)
- Tool registration via `@tool` + `ToolBox`, dispatched from the C++ registry
- Graph builder with structured `GraphResult`
- `AsyncRuntime` for concurrent steps (GIL released during provider calls)
- Mock + OpenAI + Anthropic + Ollama providers (best-effort, not battle-tested)
- Per-call timeouts + `CancelToken` on `step()` / `stream()`
- Bounded router inboxes with `Reject` / `DropOldest` / `DropNewest` policies
- Persistence — `StateStore` with in-memory + SQLite backends
- Observability — `TraceSink` with `null` / `print` / `OpenTelemetry` sinks
- Usage tracking, retry, and rate-limiting configured on the Runtime
- Embeddable pure-C++ core (see [examples/embed_cpp/](#))

What is **not** done and should not yet be relied on:

- Real-world soak testing and a bus factor ≥ 2 (the remaining gates before this is your default for a live system)
- Full multi-turn tool-use protocol (automatic tool-call loop inside step)
- Published, *measured* benchmarks vs LangGraph (the harness exists; numbers are not yet committed)
- PyPI publishing — wheels build to GitHub artifacts; trusted publishing is wired but nothing is on PyPI yet
- ABI stability across versions (source-level stability only, per [STABILITY.md](#))
- An **ARI conformance kit** — the executable proof of conformance (see [docs/ari-strategy.md](#))

Compiler (experimental)

An additive **compiler layer** turns a declaratively-defined agent graph into a portable, executable plan:

```
AgentGraph → ARI manifest (JSON) → RuntimePlan (JSON) → native
runtime → ExecutionTrace (JSON)
```

It compiles agent graphs (not arbitrary Python) without changing the runtime or the normative [ARI spec](#). The pipeline is feature-complete: real, pluggable providers (mock default; OpenAI/Anthropic/Ollama built in; bring your own); a policy-gated **tool-use loop**; governance — **policy** enforcement, **budgets** (steps/tokens/cost/wall-clock), evidence, **rollback**, deterministic **replay**, deployment manifests; native C++ JSON; and a **TypeScript frontend** ([ts/](#)) that emits identical ARI. Authoring in Python uses a mock provider with no API key.

See [docs/compiler_architecture.md](#), [docs/marrow_compiler_positioning.md](#), and the runnable [examples/python_to_ari_compile/](#)

- [examples/governance_example.py](#). It is feature-complete but **early/pre-production** — see the docs for the honest boundary.

Roadmap

- **v0.1** — Real provider tests w/ key-gated CI; ToolCall integration in the step loop; streaming benchmark vs LangGraph
 - **v0.2** — StateStore interface + SQLite impl; TraceSink for OTel; bounded inboxes
 - **v0.3** — DAG executor; async-iterator streaming API; first stable API contract
 - **v1.0** — ABI promise per module; cibuildwheel wheels on PyPI; plugin loader for C++ providers
-

License

marrow is licensed under the **Apache License, Version 2.0**. The full text is in [LICENSE](#) and required attribution is in [NOTICE](#).

Why Apache 2.0?

Picked over MIT specifically for the **explicit patent grant in §3** — contributors grant a perpetual, worldwide, royalty-free patent license to users for any patents reading on their contributions. This protects downstream commercial adopters from patent ambushes by contributors. Picked over GPL because marrow is intended to be embedded in proprietary products without forcing them to be open-sourced.

What the license permits

- **Commercial use** — including embedding in proprietary or closed-source products, no royalty
- **Modification** and creation of derivative works
- **Distribution** of source and binary forms
- **Private use** without disclosure
- **Sublicensing** under different terms (e.g. a downstream MIT package can include marrow)

What you must do

- **Preserve** the copyright and license notice in source distributions of marrow code

- **Carry forward** the contents of NOTICE in derivative distributions, where applicable
- **State modifications** if you redistribute a modified version (Apache §4(b))
- **Not use** "marrow" as the primary brand of a competing or derivative product (see *Trademark* below). The Apache license does *not* grant trademark rights.

What you do not get

- Warranty of any kind — marrow is provided "AS IS" (Apache §7)
- Liability — contributors are not liable for damages from use (Apache §8)
- Trademark license — see below

Trademark

The name "marrow" is not currently a registered trademark. We ask the community to:

- **Welcome:** use the name to describe integrations, plugins, or compatible implementations ("X for marrow", "marrow-compatible Y")
- **Avoid:** using "marrow" as the primary brand of a competing or derivative product to prevent user confusion

Patent statement

By contributing to marrow, you grant — under Apache §3 — a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the work, where such license applies only to those patent claims licensable by you that are necessarily infringed by your contribution alone or in combination with the work.

If any entity institutes patent litigation against another entity claiming the work infringes their patents, all patent licenses granted to that entity under this License terminate as of the filing date.

Contributing

Pull requests are welcome. See [CONTRIBUTING.md](#) for development setup, code style, and how submissions are licensed.

By submitting a PR you confirm — under Apache §5 — that your contribution is licensed to the project under the same Apache 2.0 terms, without additional restrictions.

Acknowledgements

- [Pybind11](#) — the seamless C++/Python interop that makes this whole project tractable.
- [scikit-build-core](#) — modern CMake-driven Python builds.
- [cibuildwheel](#) — wheels for every platform with one workflow.

Inspiration from LangGraph, CrewAI, AutoGen, and the broader open-source agent ecosystem — marrow is an alternative, not a replacement.
